Probability and Statistics Lab 2

Probability and Statistics Lab 2: random variables, and limit theorems

General comments and instructions

- The report must be prepared as *jupyter notebook along with compiled PDF*.
- For each task, include
  - the statistics obtained (like sample mean or anything else you use to complete the task) as well as histograms etc to illustrate your findings;
  - justification of your solution (e.g. refer to the corresponding theorems from probability theory); whenever applicable, provide LaTeX formulas and derivations.
  - conclusions (e.g. how reliable your answer is, does it agree with common sense expectations etc.).
- Not complying with these instructions may result in point deduction.

Work breakdown

- **Антоненко Іван** (~33 %): Task 1
- **Піх Матвій** (~33 %): Task 2, 3
- **Садовенко Софія** (~33 %): Task 4

A **team id number** referred to in tasks is the **two-digit** ordinal number of your team on the list. We keep it for reproducible calculations.

```
import random
import numpy as np
import matplotlib.pyplot as plt
from scipy import stats
from scipy.stats import ecdf
from itertools import product

team_id: int = 39

random.seed(team_id)
np.random.seed(team_id)
```

Task 1. Hamming codes

In this task, we discuss the $[7, 4]$ Hamming code and investigate its reliability. That coding system can correct single errors in the transmission of 4-bit messages and proceeds as follows:

(a) given a message $m = (a_1a_2a_3a_4)$, we first encode it to a 7-bit codeword $c = mG = (x_1x_2x_3x_4x_5x_6x_7)$, where G is a 4×7 generator matrix

(b) the codeword c is transmitted, and r is the received message

(c) r is checked for errors by calculating the syndrome vector $z := rH$, for a 7×3 parity-check matrix H

(d) if a single error has occurred in r, then the binary $z = (z_1 z_2 z_3)$ identifies the wrong bit no. $z_1 + 2z_2 + 4z_3$; thus (000) shows there was no error (or more than one), while (110) means the third bit (or more than one) got corrupted

(e) if the error was identified, then we flip the corresponding bit in r to get the corrected $r^* = (r_1^* r_2^* r_3^* r_4^* r_5^* r_6^* r_7^*)$; the decoded message is then $m^* := (r_3^* r_5^* r_6^* r_7^*)$.

The generator matrix G and the parity-check matrix H are given by

$$G := \begin{pmatrix} 1 & 1 & 1 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 1 & 0 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 \\ 1 & 1 & 0 & 1 & 0 & 0 & 1 \end{pmatrix}, \qquad H^\top := \begin{pmatrix} 1 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 \end{pmatrix}$$

GH must equal a zero-matrix modulo 2.

Assume that each bit in the transmission $c \mapsto r$ gets corrupted independently of the others with probability $p = (2 + \frac{\text{team-id-number}}{4})/100$. Your task:

1. Simulate the encoding-transmission-decoding process N times and find the estimate $\hat{p}$ of the probability p^* of correct transmission of a single message m. Comment why, for large N, $\hat{p}$ is expected to be close to p^*.
2. By estimating the standard deviation of the corresponding indicator of success by the sample standard deviation and using the CLT, find the half-length ε of the confidence interval $(\hat{p} - \varepsilon, \hat{p} + \varepsilon)$, which contains the true value p^* with probability at least 0.95. What choice of N guarantees that $\varepsilon \leq 0.03$?
3. Draw the histogram of the number $k = 0, 1, 2, 3, 4$ of errors in m^* while transmitting a 4-digit binary message. Do you think the random variable that counts the number of wrong bits in a decoded message has one of the known distributions? Justify your answer.

Solution

For team 39 the bit-flip probability is $p = \left(2 + \frac{39}{4}\right)/100 = 0.1175$.

First we check that $GH \equiv 0 \pmod 2$. Then we simulate N random messages: encode, corrupt each bit independently with probability p, compute the syndrome, flip the indicated bit, and decode $m^* = (r_3^* r_5^* r_6^* r_7^*)$.

```
G = np.array([[1, 1, 1, 0, 0, 0, 0],
              [1, 0, 0, 1, 1, 0, 0],
              [0, 1, 0, 1, 0, 1, 0],
              [1, 1, 0, 1, 0, 0, 1]])
H = np.array([[1, 0, 1, 0, 1, 0, 1],
              [0, 1, 1, 0, 0, 1, 1],
              [0, 0, 0, 1, 1, 1, 1]]).T
assert np.all((G @ H) % 2 == 0)

p = (2 + team_id / 4) / 100
DATA_BITS = [2, 4, 5, 6]


def transmit(m, p):
    """Encode, corrupt, correct and decode a batch of messages (rows of m)."""
    c = (m @ G) % 2
    errors = np.random.random(c.shape) < p
    r = (c + errors) % 2
    z = (r @ H) % 2
    wrong_bit = z @ np.array([1, 2, 4])
    r_star = r.copy()
```

```
    rows = np.nonzero(wrong_bit)[0]
    r_star[rows, wrong_bit[rows] - 1] ^= 1
    return r_star[:, DATA_BITS]


N = 100_000
m = np.random.randint(0, 2, size=(N, 4))
m_star = transmit(m, p)
success = np.all(m_star == m, axis=1)
p_hat = success.mean()

p_star, k_exact = 0.0, np.zeros(5)
zero_msg = np.zeros((1, 4), dtype=int)
for e in product([0, 1], repeat=7):
    e = np.array(e)
    prob = p ** e.sum() * (1 - p) ** (7 - e.sum())
    r = e[None, :]
    wb = int(((r @ H) % 2 @ [1, 2, 4])[0])
    r_star = r.copy()
    if wb:
        r_star[0, wb - 1] ^= 1
    k = int(r_star[0, DATA_BITS].sum())
    k_exact[k] += prob
p_star = k_exact[0]

print(f"p                          = {p:.4f}")
print(f"estimate p_hat (N={N})  = {p_hat:.5f}")
print(f"exact p*                   = {p_star:.5f}")
print(f"(1-p)^7 + 7p(1-p)^6        = {(1-p)**7 + 7*p*(1-p)**6:.5f}")
```

```
p                          = 0.1175
estimate p_hat (N=100000)  = 0.80539
exact p*                   = 0.80540
(1-p)^7 + 7p(1-p)^6        = 0.80540
```

1. Why $\hat{p} \approx p^*$. Let $I_j = \mathbb{1}\{m_j^* = m_j\}$ be the indicator that the j-th message is decoded correctly. The I_j are i.i.d. Bernoulli(p^*), so $\mathrm{E}I_j = p^*$ and $\hat{p} = \frac{1}{N}\sum_{j=1}^{N} I_j$. By the (strong) **Law of Large Numbers**, $\hat{p} \to p^*$ almost surely as $N \to \infty$, and $\operatorname{Var}\hat{p} = p^*(1-p^*)/N \to 0$.

The exact value also follows from the structure of the code. A single error is always corrected. The $[7, 4]$ Hamming code is *perfect*: the radius-1 balls around the 16 codewords cover all 2^7 words. So any pattern with ≥ 2 errors is decoded to a **different** codeword, and since $m \mapsto mG$ is a bijection, the message is then wrong. Hence

$$p^* = (1-p)^7 + 7p(1-p)^6 \approx 0.8054,$$

which agrees with the enumeration over all 2^7 error patterns and with the simulated $\hat{p}$ to four decimals.

```
s = success.std(ddof=1)
z95 = stats.norm.ppf(0.975)
eps = z95 * s / np.sqrt(N)
print(f"sample st. dev. s = {s:.4f}")
print(f"95% CI: ({p_hat - eps:.5f}, {p_hat + eps:.5f}),  half-length eps = {eps:.5f}")
print(f"p* inside CI: {p_hat - eps < p_star < p_hat + eps}")

N_needed = int(np.ceil((z95 * s / 0.03) ** 2))
N_worst = int(np.ceil((z95 * 0.5 / 0.03) ** 2))
```

```
print(f"N for eps <= 0.03 with s = {s:.3f}: N >= {N_needed}")
print(f"N for eps <= 0.03 in the worst case s = 0.5: N >= {N_worst}")
```

```
sample st. dev. s = 0.3959
95% CI: (0.80294, 0.80784),  half-length eps = 0.00245
p* inside CI: True
N for eps <= 0.03 with s = 0.396: N >= 670
N for eps <= 0.03 in the worst case s = 0.5: N >= 1068
```

2. Confidence interval. By the **CLT**, $\sqrt{N}(\hat{p}-p^*)/\sigma \Rightarrow \mathcal{N}(0,1)$, where $\sigma^2 = p^*(1-p^*)$. Replacing σ by the sample standard deviation s (which is consistent by the LLN, so Slutsky's lemma applies) gives

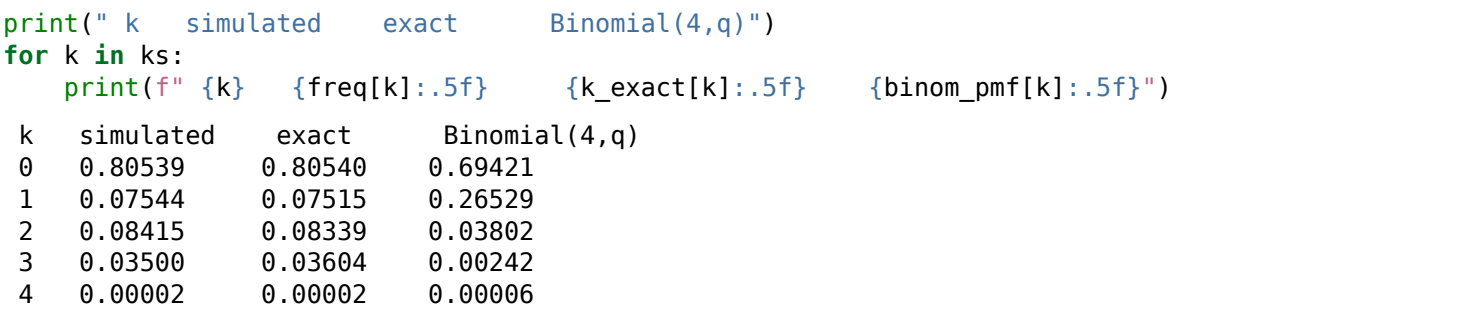
$$\mathrm{P}\left(|\hat{p}-p^*| < z_{0.975}\frac{s}{\sqrt{N}}\right) \approx 0.95, \qquad \varepsilon = z_{0.975}\frac{s}{\sqrt{N}} = 1.96\,\frac{s}{\sqrt{N}}.$$

For $\varepsilon \le 0.03$ we need $N \ge \left(\dfrac{1.96\,s}{0.03}\right)^2$. With our s this is the N printed above (a few hundred). Since $s \le 1/2$ for any Bernoulli variable, $N \ge 1068$ is enough whatever p^* is.

```
k_errors = np.sum(m_star != m, axis=1)
freq = np.bincount(k_errors, minlength=5) / N

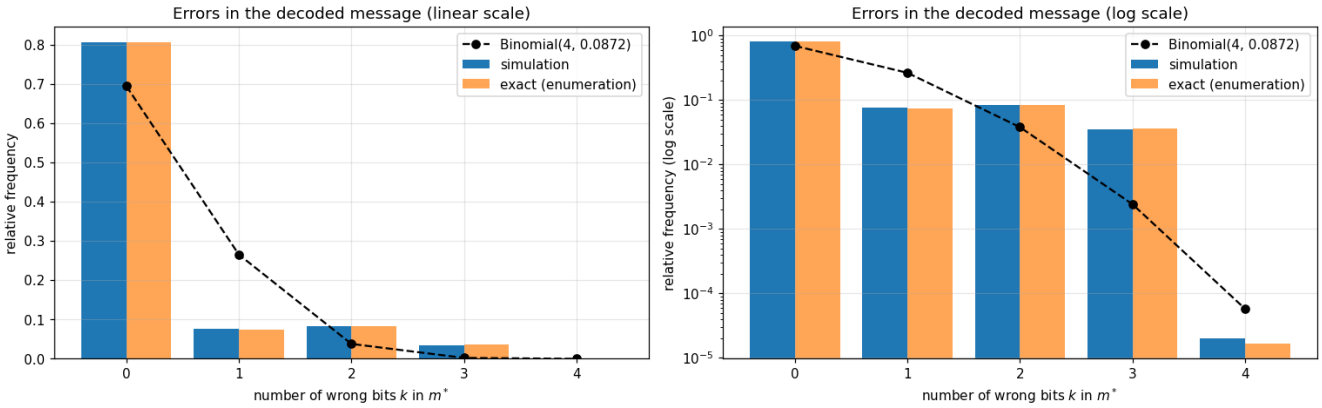
q = k_errors.mean() / 4
binom_pmf = stats.binom.pmf(np.arange(5), 4, q)

ks = np.arange(5)
fig, axes = plt.subplots(1, 2, figsize=(14, 4.5))
for ax, log in zip(axes, [False, True]):
    ax.bar(ks - 0.2, freq, width=0.4, label="simulation")
    ax.bar(ks + 0.2, k_exact, width=0.4, alpha=0.7, label="exact (enumeration)")
    ax.plot(ks, binom_pmf, "ko--", label=f"Binomial(4, {q:.4f})")
    if log:
        ax.set_yscale("log")
    ax.set_xlabel("number of wrong bits $k$ in $m^*$")
    ax.set_ylabel("relative frequency" + (" (log scale)" if log else ""))
    ax.set_title("Errors in the decoded message" + (" (log scale)" if log else " (linear scale)"))
    ax.set_xticks(ks)
    ax.legend()
    ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



```
print(" k   simulated    exact      Binomial(4,q)")
for k in ks:
    print(f" {k}   {freq[k]:.5f}     {k_exact[k]:.5f}    {binom_pmf[k]:.5f}")
```

```
 k   simulated    exact      Binomial(4,q)
 0   0.80539     0.80540    0.69421
 1   0.07544     0.07515    0.26529
 2   0.08415     0.08339    0.03802
 3   0.03500     0.03604    0.00242
 4   0.00002     0.00002    0.00006
```



3. Distribution of the number of wrong bits. The number of wrong bits in m^* is **not** one of the standard distributions; in particular it is not binomial. It is a sum of four Bernoulli indicators (one per data bit), but those indicators are **dependent**. Decoding errors happen in groups: two channel errors produce three wrong bits in r^*, which often include two or three data bits at once. Compared with $\mathrm{Bin}(4, q)$ of the same mean, the true distribution puts noticeably more mass on $k = 0$ and on $k = 2, 3$ and much less on $k = 1$ (for $k = 4$ both probabilities are negligible, about 10^{-5}) (see the table and the log-scale plot). The exact law is the finite distribution obtained by enumerating error patterns, and the simulation matches it.

Task 2. Particle decay

In this task, we discuss a real-life process that is well modelled by a Poisson distribution. As you remember, a Poisson random variable describes occurrences of rare events, i.e., counts the number of successes in a large number of independent random experiments. One of the typical examples is the radioactive decay process.

Consider a sample of radioactive element of mass m, which has a large half-life period T; it is vitally important to know the probability that during a one second period, the number of nuclei decays will not exceed some critical level k. This probability can easily be estimated using the fact that, given the decay constant λ of the element (i.e., the probability that a given nucleus decays within one second) and the number N of atoms in the sample, the random number of decays within a second is well modelled by Poisson distribution with parameter $\mu := N\lambda$. Next, for the sample of mass m, the number of atoms is $N = \frac{m}{M} N_A$, where $N_A = 6 \times 10^{23}$ is the Avogadro constant, and M is the molar (atomic) mass of the element. The decay constant, λ, is $\ln(2)/T$, where T is measured in seconds.

Assume that a medical laboratory receives n samples of radioactive element ^{137}Cs (used in radiotherapy) with half-life period $T = 30.1$ years and mass $m = (10 + \text{team-id-number}) \times 10^{-14}$ g each. Denote by $X_1, X_2, \dots, X_n$ the i.i.d. r.v.'s counting the number of decays in sample i in one second.

1. Specify the parameter of the Poisson distribution of X_i (you'll need the atomic mass of Cesium-137)
2. Show that the distribution of the sample means of $X_1, \dots, X_n$ gets very close to a normal one as n becomes large and identify that normal distribution. To this end,
   - simulate the realization $x_1, x_2, \dots, x_n$ of the X_i and calculate the sample mean $s = \bar{x}$;
   - repeat this K times to get the sample $\mathbf{s} = (s_1, \dots, s_K)$ of means and form the empirical cumulative distribution function $\widehat{F}_{\mathbf{s}}$ of $\mathbf{s}$;
   - identify μ and σ^2 such that the c.d.f. F of $\mathcal{N}(\mu, \sigma^2)$ is close to the e.c.d.f. $\widehat{F}_{\mathbf{s}}$ and plot both c.d.f.'s on one graph to visualize their proximity (use the proper scales!);
   - calculate the maximal difference between the two c.d.f.'s;
   - consider cases $n = 5$, $n = 10$, $n = 50$ and comment on the results.
3. Calculate the largest possible value of n, for which the total number of decays in one second is less than $a := 100$ with probability more than 0.95. To this end,

- simulate the realization $x_1, x_2, \dots, x_n$ of the X_i and calculate the sum $s = x_1 + \cdots + x_n$;
- repeat this K times to get the sample $\mathbf{s} = (s_1, \dots, s_K)$ of sums;
- calculate the number of elements of the sample which are less than critical value (a) and calculate the empirical probability;
- obtain the theoretical bound using Markov inequality, Chernoff bound and Central Limit Theorem, and compare the results.

Solution

1. Poisson parameter. The atomic mass of ^{137}Cs is $M \approx 136.907$ g/mol and $m = (10+39)\cdot 10^{-14} = 4.9 \cdot 10^{-13}$ g. Then

$$N = \frac{m}{M} N_A, \qquad \lambda = \frac{\ln 2}{T}, \quad T = 30.1 \cdot 365.25 \cdot 24 \cdot 3600 \text{ s}, \qquad \mu = N\lambda.$$

```
N_A = 6e23
M_Cs = 136.907
T_half = 30.1 * 365.25 * 24 * 3600
mass = (10 + team_id) * 1e-14

N_atoms = mass / M_Cs * N_A
lam = np.log(2) / T_half
mu = N_atoms * lam

print(f"N atoms     = {N_atoms:.4e}")
print(f"lambda      = {lam:.4e} 1/s")
print(f"mu = N*lam  = {mu:.4f} decays per second")
```

```
N atoms     = 2.1474e+09
lambda      = 7.2972e-10 1/s
mu = N*lam  = 1.5670 decays per second
```

So $X_i \sim \text{Poisson}(\mu)$ with $\mu \approx 1.567$.

2. CLT for sample means. $\mathsf{E}X_i = \operatorname{Var} X_i = \mu$, so by the CLT

$$\bar{X}_n \approx \mathcal{N}\left(\mu, \frac{\mu}{n}\right).$$

We take $K = 10^4$ samples of means for $n = 5, 10, 50$ and compare the e.c.d.f. with this normal c.d.f.

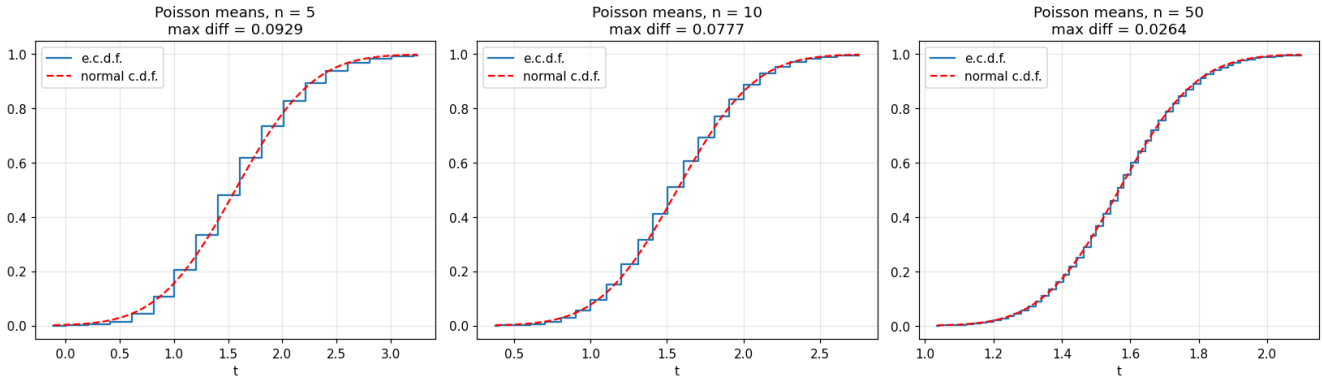
```
K = 10_000


def compare_with_normal(s, mu_, sigma_, title, ax):
    Fs = ecdf(s).cdf.evaluate
    Fz = stats.norm(mu_, sigma_).cdf
    t = np.linspace(mu_ - 3 * sigma_, mu_ + 3 * sigma_, 200)
    diff = np.max(np.abs(Fs(t) - Fz(t)))
    ax.step(t, Fs(t), where="post", label="e.c.d.f.")
    ax.plot(t, Fz(t), "r--", label="normal c.d.f.")
    ax.set_title(f"{title}\nmax diff = {diff:.4f}")
    ax.set_xlabel("t")
    ax.legend()
    ax.grid(alpha=0.3)
    return diff


fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
```

```
for ax, n in zip(axes, [5, 10, 50]):
    s = np.random.poisson(mu, size=(K, n)).mean(axis=1)
    d = compare_with_normal(s, mu, np.sqrt(mu / n),
                            f"Poisson means, n = {n}", ax)
    print(f"n = {n:2d}: N({mu:.3f}, {mu/n:.4f}),  max |F_s - F| = {d:.4f}")
plt.tight_layout()
plt.show()
```

```
n =  5: N(1.567, 0.3134),  max |F_s - F| = 0.0929
n = 10: N(1.567, 0.1567),  max |F_s - F| = 0.0777
n = 50: N(1.567, 0.0313),  max |F_s - F| = 0.0264
```



Comments. As n grows the e.c.d.f. of the sample means approaches the c.d.f. of $\mathcal{N}(\mu, \mu/n)$ and the maximal difference decreases, as the CLT says. Two effects slow the convergence down:

- **Discreteness.** $n\bar{X}_n$ is integer-valued, so $\hat{F}_{\mathbf{s}}$ is a step function with jumps of size about $1/\sqrt{2\pi n\mu}$. The distance to a continuous c.d.f. cannot be smaller than roughly half a jump, and this term is the main one for small n.
- **Skewness.** The Poisson law has skewness $1/\sqrt{\mu}$, which makes the Berry–Esseen error of order $1/\sqrt{n\mu}$.

Both terms decay like $1/\sqrt{n}$, which matches the decreasing maximal differences for $n = 5, 10, 50$.

3. Largest n with $\mathsf{P}(S_n < 100) > 0.95$. Here $S_n = X_1 + \cdots + X_n \sim \text{Poisson}(n\mu)$, so the event is $\{S_n \le 99\}$ and we need $\mathsf{P}(S_n \ge 100) < 0.05$.

- **Markov:** $\mathsf{P}(S_n \ge a) \le \dfrac{\mathsf{E}S_n}{a} = \dfrac{n\mu}{100}$, so it suffices that $n\mu/100 \le 0.05$, i.e. $n \le 5/\mu$.
- **Chernoff:** for $t > 0$, $\mathsf{P}(S_n \ge a) \le e^{-ta}\,\mathsf{E}e^{tS_n} = \exp\left(-ta + n\mu(e^t - 1)\right)$. Minimising at $t = \ln \frac{a}{n\mu}$ gives

$$\mathsf{P}(S_n \ge a) \le e^{-n\mu}\left(\frac{e\,n\mu}{a}\right)^a, \quad a > n\mu.$$

- **CLT:** $S_n \approx \mathcal{N}(n\mu, n\mu)$, so (with continuity correction) $\mathsf{P}(S_n < 100) \approx \Phi\left(\dfrac{99.5 - n\mu}{\sqrt{n\mu}}\right) > 0.95$.

Markov and Chernoff give *guaranteed* (sufficient) values of n. The CLT gives an approximation, and the simulation gives an empirical estimate.

```
a = 100
ns = np.arange(1, 81)

emp, markov, chernoff, clt, exact = [], [], [], [], []
for n in ns:
    sums = np.random.poisson(mu, size=(K, n)).sum(axis=1)
    emp.append(np.mean(sums < a))
```

```
    lam_n = n * mu
    markov.append(min(1, lam_n / a))
    chernoff.append(np.exp(-lam_n + a * np.log(np.e * lam_n / a)) if lam_n < a else 1)
    clt.append(stats.norm.sf((a - 0.5 - lam_n) / np.sqrt(lam_n)))
    exact.append(stats.poisson.sf(a - 1, lam_n))

emp, markov, chernoff, clt, exact = map(np.array, (emp, markov, chernoff, clt, exact))


def max_n(ok):
    return ns[ok].max() if ok.any() else 0

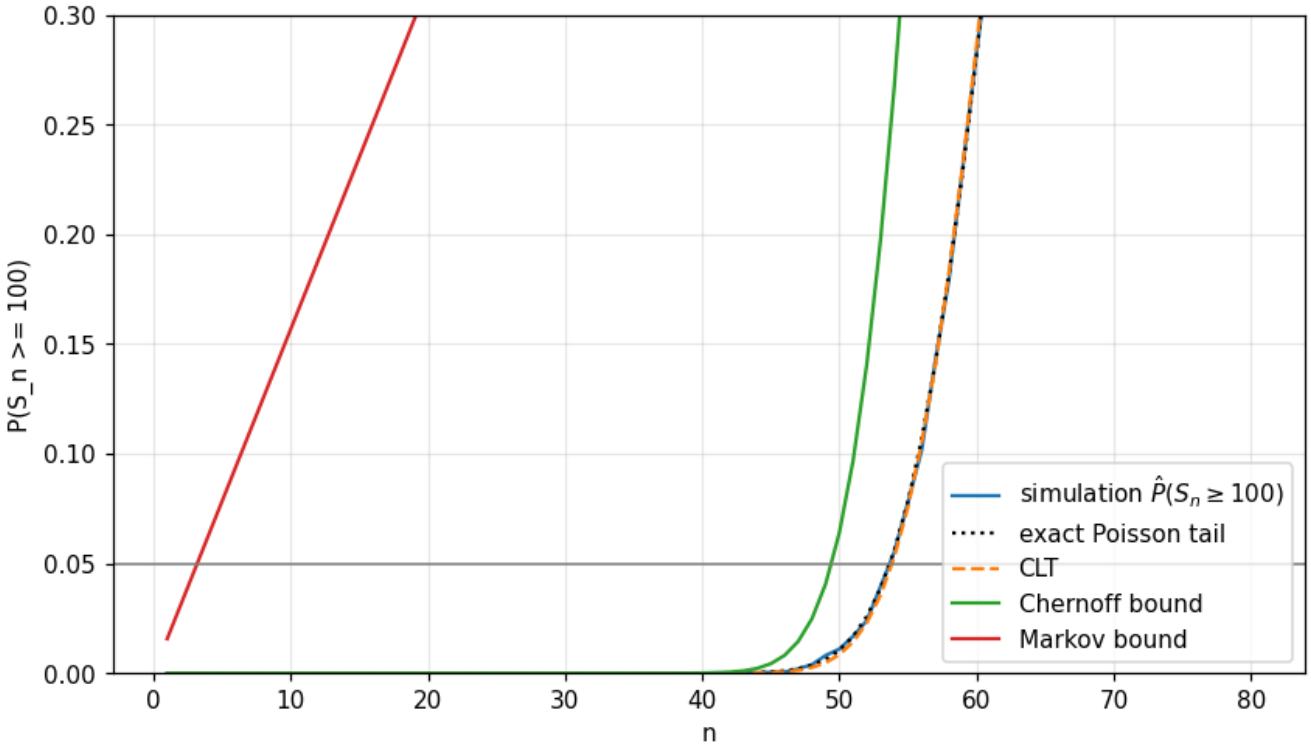

results = {
    "Markov bound":      max_n(markov <= 0.05),
    "Chernoff bound":    max_n(chernoff <= 0.05),
    "CLT approximation": max_n(clt < 0.05),
    "Simulation":        max_n(emp > 0.95),
    "Exact (Poisson)":   max_n(exact < 0.05),
}
for k, v in results.items():
    print(f"{k:18s}: n_max = {v}")

n_sim = results["Simulation"]
print(f"\nempirical P(S < 100) for n = {n_sim}: {emp[n_sim-1]:.4f},"
      f" for n = {n_sim+1}: {emp[n_sim]:.4f}")

plt.figure(figsize=(9, 5))
plt.plot(ns, 1 - emp, label="simulation $\\hat P(S_n \\geq 100)$")
plt.plot(ns, exact, "k:", label="exact Poisson tail")
plt.plot(ns, clt, "--", label="CLT")
plt.plot(ns, chernoff, label="Chernoff bound")
plt.plot(ns, markov, label="Markov bound")
plt.axhline(0.05, color="gray", lw=1)
plt.ylim(0, 0.3)
plt.xlabel("n")
plt.ylabel("P(S_n >= 100)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

```
Markov bound      : n_max = 3
Chernoff bound    : n_max = 49
CLT approximation : n_max = 53
Simulation        : n_max = 53
Exact (Poisson)   : n_max = 53

empirical P(S < 100) for n = 53: 0.9605, for n = 54: 0.9448
```



Conclusions. All methods agree on the *direction*, but they differ a lot in sharpness:

- **Markov** uses only the mean, so it is very conservative and allows only $n = 3$.
- **Chernoff** uses the whole moment generating function, so it is far tighter. It is still a guaranteed upper bound, so it gives a smaller n than the true one.
- **CLT** gives almost the exact answer, since S_n is Poisson with a large parameter (≈ 85) and therefore close to normal. The exact Poisson computation and the simulation confirm the value.

The simulated threshold can differ by one from the exact one because $K = 10^4$ simulations estimate a probability with standard error of about 0.002, and close to the threshold that is enough to tip the comparison.

Task 3. Geiger counter

In this task, we use the Central Limit Theorem approximation for continuous random variables.

One of the devices to measure radioactivity level at a given location is the Geiger counter. When the radioactive level is almost constant, the time between two consecutive clicks of the Geiger counter is an exponentially distributed random variable with parameter $\nu_1 = \frac{\text{team-id-number}}{10} + 1$ (minutes^{-1}). Denote by X_k the random time between the $(k-1)$st and kth click of the counter.

1. Show that the distribution of the sample means of $X_1, X_2, \dots, X_n$ gets very close to a normal one (which one?) as n becomes large. To this end,
   - simulate the realizations $x_1, x_2, \dots, x_n$ of the r.v. X_i and calculate the sample mean $s = \bar{x}$;
   - repeat this K times to get the sample $\mathbf{s} = (s_1, \dots, s_K)$ of means and then the empirical cumulative distribution function $F_{\mathbf{s}}$ of $\mathbf{s}$;
   - identify μ and σ^2 such that the c.d.f. of $\mathcal{N}(\mu, \sigma^2)$ is close to the e.c.d.f. $F_{\mathbf{s}}$ and plot both c.d.f.'s on one graph to visualize their proximity;
   - calculate the maximal difference between the two c.d.f.'s;
   - consider cases $n = 5$, $n = 10$, $n = 50$ and comment on the results.

2. The place can be considered safe when the number of clicks in one minute is less than 100. It is known that the parameter ν is proportional to the number N of the radioactive samples, i.e., $\nu = N\nu_1$. Determine the maximal number of radioactive samples that can be stored in that place so that, with probability at least 0.95, the place is identified as safe. To do this,

   (a) express the event of interest in terms of the r.v. $S := X_1 + \cdots + X_{100}$;

   (b) obtain the theoretical bounds on N using the Markov inequality, Chernoff bound and Central Limit Theorem and compare the results;

   (c) with the predicted N and thus ν, simulate the realization $x_1, x_2, \dots, x_{100}$ of the X_i and of the sum $S = X_1 + \cdots + X_{100}$;

   (d) repeat this K times to get the sample $\mathbf{s} = (s_1, \dots, s_K)$ of total times until the 100th click;

   (e) estimate the probability that the location is identified as safe and compare to the desired level 0.95

Solution

For team 39, $\nu_1 = 39/10 + 1 = 4.9$ min^{-1}, and $X_k \sim \mathcal{E}(\nu_1)$ with $\mathsf{E}X_k = 1/\nu_1$ and $\operatorname{Var} X_k = 1/\nu_1^2$.

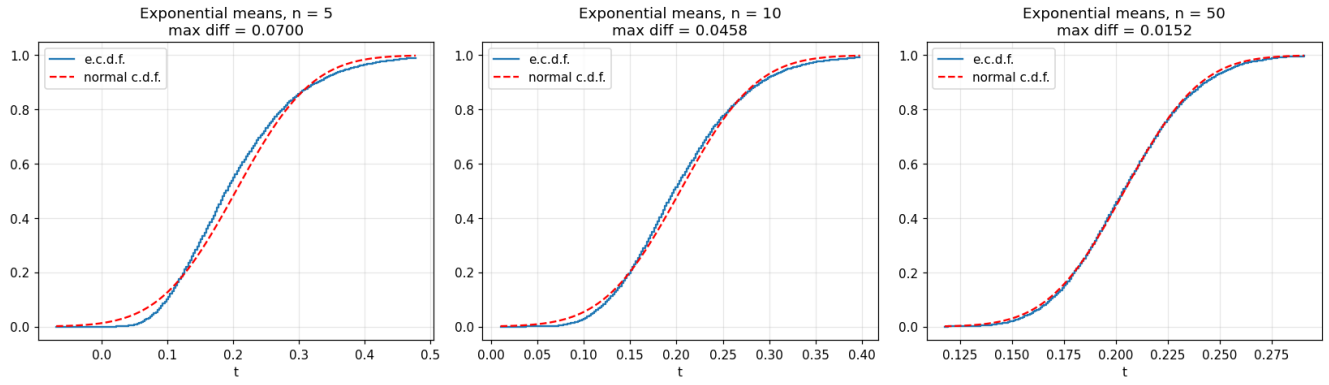
1. CLT for sample means. By the CLT,

$$\bar{X}_n \approx \mathcal{N}\left(\frac{1}{\nu_1}, \frac{1}{n\nu_1^2}\right).$$

```
nu1 = team_id / 10 + 1
K = 10_000

fig, axes = plt.subplots(1, 3, figsize=(15, 4.5))
for ax, n in zip(axes, [5, 10, 50]):
    s = np.random.exponential(scale=1 / nu1, size=(K, n)).mean(axis=1)
    mu_, sigma_ = 1 / nu1, 1 / (nu1 * np.sqrt(n))
    d = compare_with_normal(s, mu_, sigma_, f"Exponential means, n = {n}", ax)
    print(f"n = {n:2d}: N({mu_:.4f}, {sigma_**2:.6f}),  max |F_s - F| = {d:.4f}")
plt.tight_layout()
plt.show()
```

```
n =  5: N(0.2041, 0.008330),  max |F_s - F| = 0.0700
n = 10: N(0.2041, 0.004165),  max |F_s - F| = 0.0458
n = 50: N(0.2041, 0.000833),  max |F_s - F| = 0.0152
```



Comments. The e.c.d.f. of the means gets closer to $\mathcal{N}(1/\nu_1, 1/(n\nu_1^2))$ as n grows. The remaining discrepancy comes from the skewness of the exponential law (skewness 2). For n exponentials the sample mean is exactly $\Gamma(n, n\nu_1)$, whose skewness $2/\sqrt{n}$ goes to 0, and by Berry–Esseen the c.d.f. distance is $O(1/\sqrt{n})$. Unlike Task 2 there is no discreteness, so the differences are smaller.

2. Safe storage.

(a) Fewer than 100 clicks in one minute means that the 100th click comes later than one minute:

$$\{\text{safe}\} = \{S > 1\}, \qquad S = X_1 + \cdots + X_{100}, \quad X_k \sim \mathcal{E}(\nu),\ \nu = N\nu_1.$$

Then $S \sim \Gamma(100, \nu)$, with $\mathsf{E}S = 100/\nu$ and $\operatorname{Var} S = 100/\nu^2$. We need $\mathsf{P}(S \le 1) \le 0.05$.

(b) Bounds on N.

- **Markov:** Markov bounds only upper tails: $\mathsf{P}(S \ge 1) \le \mathsf{E}S = 100/\nu$. For $\mathsf{P}(S > 1) \ge 0.95$ it is *necessary* that $100/\nu \ge 0.95$, i.e. $\nu \le 105.3$ and $N \le 21$. This is only an upper limit; it does not guarantee safety. Equivalently, the number of clicks per minute $C \sim \text{Poisson}(\nu)$, and Markov $\mathsf{P}(C \ge 100) \le \nu/100 \le 0.05$ *guarantees* safety for $\nu \le 5$, i.e. only $N = 1$.
- **Chernoff (lower tail):** for $t > 0$,

$$\mathsf{P}(S \le 1) = \mathsf{P}(e^{-tS} \ge e^{-t}) \le e^t\, \mathsf{E}e^{-tS} = e^t \left(\frac{\nu}{\nu + t}\right)^{100}.$$

  The minimum is at $t = 100 - \nu$, which gives $\mathsf{P}(S \le 1) \le e^{100-\nu} \left(\frac{\nu}{100}\right)^{100}$. We need this ≤ 0.05; it is a *sufficient* condition.
- **CLT:** $S \approx \mathcal{N}(100/\nu, 100/\nu^2)$, so

$$\mathsf{P}(S > 1) \approx \Phi\left(\frac{100/\nu - 1}{10/\nu}\right) = \Phi\left(\frac{100 - \nu}{10}\right) \ge 0.95 \iff \nu \le 100 - 10 z_{0.95} \approx 83.55.$$

```
z95_one = stats.norm.ppf(0.95)
Ns = np.arange(1, 31)
nus = Ns * nu1

markov_nec = 100 / nus >= 0.95
markov_suf = nus / 100 <= 0.05
chernoff_ok = np.where(nus < 100,
                       np.exp(100 - nus + 100 * np.log(nus / 100)), 1) <= 0.05
clt_ok = stats.norm.cdf((100 - nus) / 10) >= 0.95
exact_ok = stats.gamma.sf(1, a=100, scale=1 / nus) >= 0.95

print(f"nu1 = {nu1}")
print(f"Markov (necessary)    : N <= {Ns[markov_nec].max()}")
print(f"Markov (sufficient)   : N <= {Ns[markov_suf].max() if markov_suf.any() else 0}")
print(f"Chernoff (sufficient) : N <= {Ns[chernoff_ok].max()}")
print(f"CLT                   : N <= {Ns[clt_ok].max()}   (nu <= {100 - 10*z95_one:.2f})")
print(f"Exact (Gamma)         : N <= {Ns[exact_ok].max()}")
```

```
nu1 = 4.9
Markov (necessary)    : N <= 21
Markov (sufficient)   : N <= 1
Chernoff (sufficient) : N <= 15
CLT                   : N <= 17   (nu <= 83.55)
Exact (Gamma)         : N <= 17
```

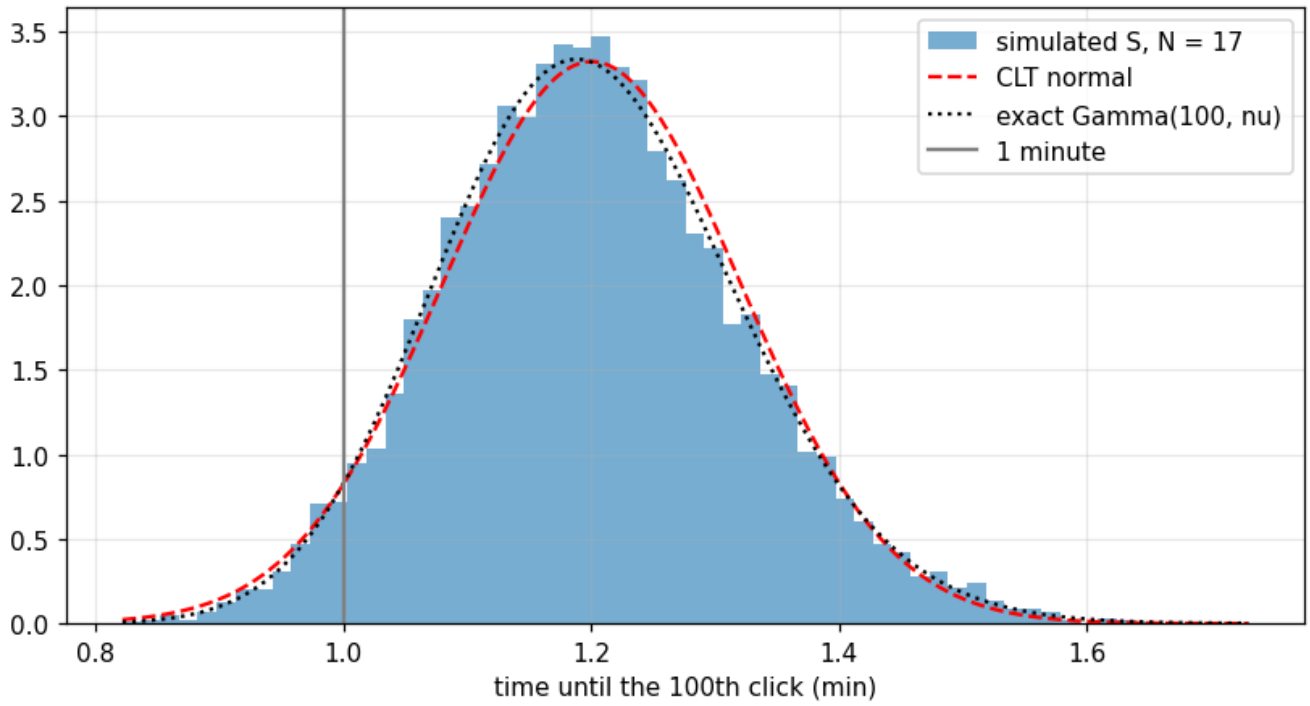
(c)-(e) We simulate $K = 10^4$ sums S of 100 exponential inter-click times with the CLT-predicted N (and with $N + 1$ for comparison) and estimate $\mathsf{P}(S > 1)$.

```
N_pred = Ns[clt_ok].max()
for N_ in [N_pred, N_pred + 1]:
    nu = N_ * nu1
    S = np.random.exponential(scale=1 / nu, size=(K, 100)).sum(axis=1)
    p_safe = np.mean(S > 1)
    se = np.sqrt(p_safe * (1 - p_safe) / K)
```

```
    print(f"N = {N_}, nu = {nu:.1f}: estimated P(safe) = {p_safe:.4f} "
          f"(+- {1.96*se:.4f}),  exact = {stats.gamma.sf(1, 100, scale=1/nu):.4f}")

nu = N_pred * nu1
S = np.random.exponential(scale=1 / nu, size=(K, 100)).sum(axis=1)
plt.figure(figsize=(9, 4.5))
plt.hist(S, bins=60, density=True, alpha=0.6, label=f"simulated S, N = {N_pred}")
x = np.linspace(S.min(), S.max(), 300)
plt.plot(x, stats.norm.pdf(x, 100 / nu, 10 / nu), "r--", label="CLT normal")
plt.plot(x, stats.gamma.pdf(x, 100, scale=1 / nu), "k:", label="exact Gamma(100, nu)")
plt.axvline(1, color="gray", label="1 minute")
plt.xlabel("time until the 100th click (min)")
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

```
N = 17, nu = 83.3: estimated P(safe) = 0.9594 (+- 0.0039),  exact = 0.9590
N = 18, nu = 88.2: estimated P(safe) = 0.8898 (+- 0.0061),  exact = 0.8842
```



Conclusions. With the CLT choice $N = 17$ ($\nu = 83.3$) the estimated probability that the location is classified as safe is about 0.95, as required. With $N = 18$ it drops clearly below 0.95. The exact Gamma computation confirms $N_{\max} = 17$.

The Chernoff bound guarantees safety only up to a smaller N; it is a rigorous but conservative bound. Markov is far too weak in its "sufficient" form ($N = 1$), and in its "necessary" form it only tells us $N \leq 21$. As in Task 2, the CLT is the most accurate tool here because S is a sum of 100 i.i.d. terms, while Markov and Chernoff are worst-case inequalities.

Task 4. Expectation fallacy

This task consists of two parts:

1. In this part, we discuss independence of random variables and its moments: expectation and variance.

(a) Suppose we have a random variable X. Explain why $E\left(\frac{1}{X}\right) \neq \frac{1}{E(X)}$;

(b) Let $X \sim \mathcal{N}(\mu, \sigma^2)$ with $\mu = 5$ and $\sigma^2 = 10$. Simulate realizations $x_1, x_2, \dots, x_{100}$ of X and $y_1, y_2, \dots, y_{100}$ of $Y := \frac{1}{X}$ to calculate the values of $\frac{1}{\bar{X}}$ and $\bar{Y}$. Comment on the received results;

(c) Let X and Y be independent exponentially distributed r.v.'s with parameter $\lambda = 2$. Set $Z := \ln X + 5$. Plot the Quantile-Quantile plot and scatterplot of X and Y. Plot the Quantile-Quantile plot and scatterplot of X and Z. Explain the results. Comment on the difference of relations between the pairs of random variables. Which pair of r.v.'s is dependent and which one is identically distributed?

2. You toss a fair coin three times and a random variable X records how many times the coin shows Heads. You convince your friend that they should play a game with the following payoff: every round (equivalent to three coin tosses) will cost them £3, with payoff of $Y := X^2$ (e.g., £4 for two Heads). Your friend argues: "on average there are 1.5 Heads, so I win £$1.5^2 = $ £2.25 < £3, hence the game is unfair". Is the game fair, i.e., is the expected value of the net gain zero? What is the variance of Y? To answer this,

   (a) Explain what type of random variable is X:

   - Normally distributed
   - Binomially distributed
   - Poisson distributed
   - Uniformly distributed

   (b) What are the expected value and variance of X? Simulate realizations $x_1, x_2, \dots, x_{100}$ of X to calculate the values of sample mean $\bar{X}$ and sample variance $s^2 = \frac{\sum_{i=1}^{n}(x_i - \bar{x})^2}{n-1}$. Comment on the results;

   (c) Is your friend's argument correct? Relate it to part 1(a). Compute the expected value and variance of Y exactly. Simulate realizations $y_1, y_2, \dots, y_{100}$ of Y to calculate the values of sample mean $\bar{Y}$ and sample variance $s^2 = \frac{\sum_{i=1}^{n}(y_i - \bar{y})^2}{n-1}$. Comment on the results;

   (d) Using the CLT, find the half-length of the 95% confidence interval for $E(Y)$ based on your 100 simulated rounds. Could the simulation alone tell you whether the game is fair? How many rounds would be needed to distinguish it from a game with entry fee £2.9?

Solution

1(a). The expectation is linear, $\mathsf{E}(aX + b) = a\mathsf{E}X + b$, but $g(x) = 1/x$ is not linear, and in general $\mathsf{E}\, g(X) \neq g(\mathsf{E}X)$. For a positive X the function $1/x$ is strictly convex, so by **Jensen's inequality** $\mathsf{E}(1/X) \geq 1/\mathsf{E}X$, with equality only when X is constant. For example, if X takes the values 1 and 3 with probability $1/2$ each, then $1/\mathsf{E}X = 1/2$ but $\mathsf{E}(1/X) = \frac{1}{2}(1 + \frac{1}{3}) = \frac{2}{3}$. If X can be near 0, $\mathsf{E}(1/X)$ may not exist at all. That is the case for a normal X: its density is positive at 0, and $\int \frac{1}{|x|} f(x)\, dx = \infty$.

```
mu_x, var_x = 5, 10
x = np.random.normal(mu_x, np.sqrt(var_x), 100)
y = 1 / x
print(f"1 / x_bar = {1 / x.mean():.4f}   (1/mu = {1/mu_x})")
print(f"y_bar     = {y.mean():.4f}")

reps = np.random.normal(mu_x, np.sqrt(var_x), size=(10, 100))
print("\n1/x_bar over 10 repetitions:", np.round(1 / reps.mean(axis=1), 3))
print("y_bar   over 10 repetitions:", np.round((1 / reps).mean(axis=1), 3))
```

```
1 / x_bar = 0.1986   (1/mu = 0.2)
```

```
y_bar     = 0.2806

1/x_bar over 10 repetitions: [0.205 0.19  0.202 0.194 0.217 0.225 0.227 0.223 0.205 0.198]
y_bar     over  10  repetitions:  [  0.305    0.203    0.118  -0.335    0.755    0.25    -
0.231  0.116  0.264  0.259]
```

1(b). By the LLN and continuity of $1/x$ at $\mu = 5$, $1/\bar{X} \to 1/\mu = 0.2$, and indeed $1/\bar{x}$ is close to 0.2 and stable between repetitions. $\bar{Y}$ is different: $\mathsf{E}(1/X)$ does not exist (see 1(a)), so the LLN does not apply to Y. Occasional values of x close to 0 give huge $|y|$, so $\bar{y}$ jumps around between repetitions and does not settle near 0.2. This shows $\mathsf{E}(1/X) \neq 1/\mathsf{E}X$ empirically.

```
lam2 = 2
n4 = 1000
X = np.random.exponential(scale=1 / lam2, size=n4)
Y = np.random.exponential(scale=1 / lam2, size=n4)
Z = np.log(X) + 5

fig, axes = plt.subplots(2, 2, figsize=(11, 9))
lim = max(X.max(), Y.max())
axes[0, 0].plot(np.sort(X), np.sort(Y), ".", ms=3)
axes[0, 0].plot([0, lim], [0, lim], "r--", label="y = x")
axes[0, 0].set(title="Q-Q plot: X vs Y", xlabel="quantiles of X", ylabel="quantiles of Y")
axes[0, 0].legend()

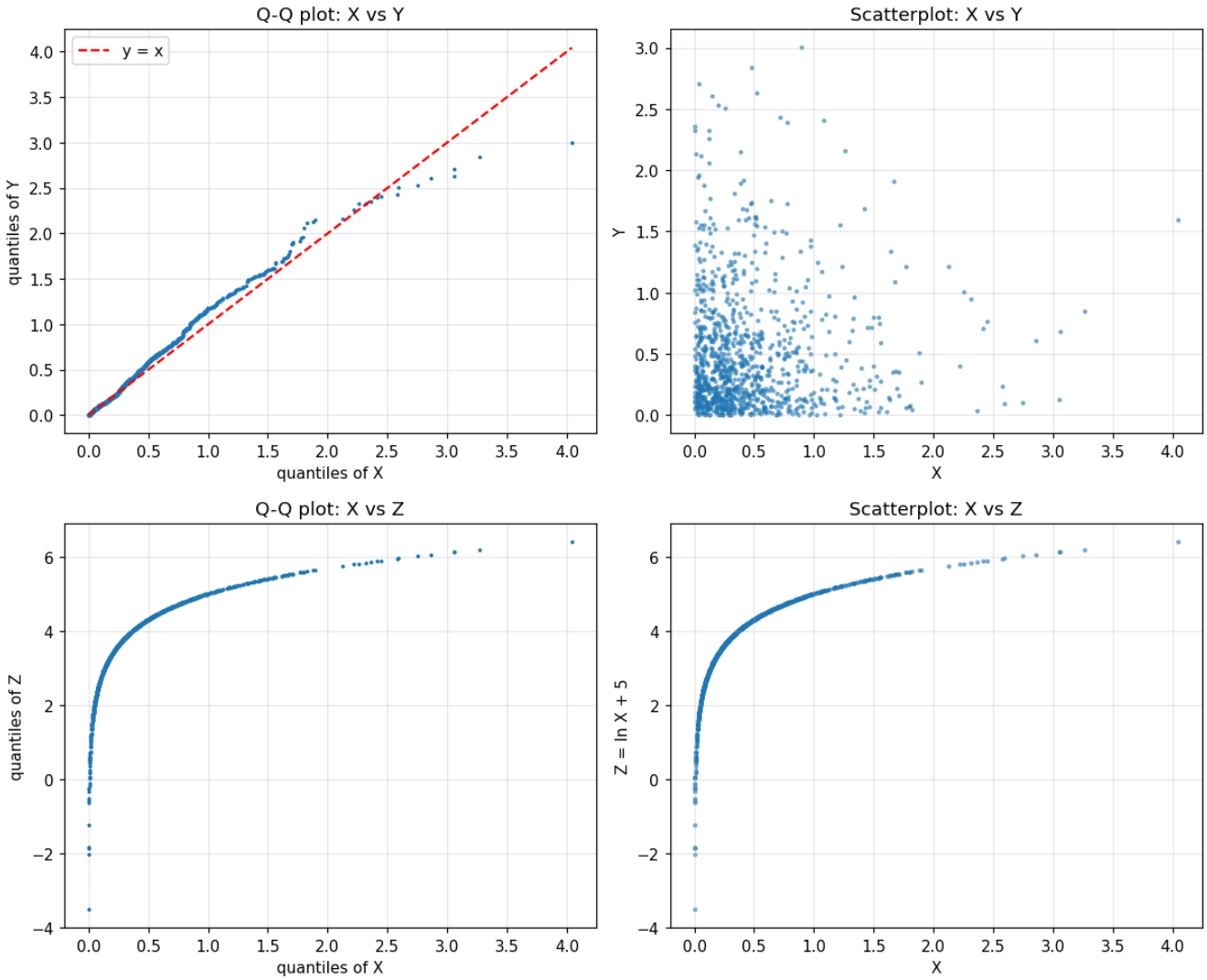
axes[0, 1].scatter(X, Y, s=4, alpha=0.5)
axes[0, 1].set(title="Scatterplot: X vs Y", xlabel="X", ylabel="Y")

axes[1, 0].plot(np.sort(X), np.sort(Z), ".", ms=3)
axes[1, 0].set(title="Q-Q plot: X vs Z", xlabel="quantiles of X", ylabel="quantiles of Z")

axes[1, 1].scatter(X, Z, s=4, alpha=0.5)
axes[1, 1].set(title="Scatterplot: X vs Z", xlabel="X", ylabel="Z = ln X + 5")
for ax in axes.flat:
    ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()

print(f"corr(X, Y) = {np.corrcoef(X, Y)[0, 1]:.4f}")
print(f"corr(X, Z) = {np.corrcoef(X, Z)[0, 1]:.4f},  "
      f"Spearman(X, Z) = {stats.spearmanr(X, Z)[0]:.4f}")
```

```
corr(X, Y) = 0.0032
corr(X, Z) = 0.7622,  Spearman(X, Z) = 1.0000
```



1(c). A Q-Q plot compares *marginal distributions*; a scatterplot shows the *joint* behaviour, i.e. dependence.

- X **and** Y**:** the Q-Q points lie on the line $y = x$, so X and Y are **identically distributed** (both $\mathcal{E}(2)$). The scatterplot is a structureless cloud (correlation ≈ 0), which is consistent with **independence**.
- X **and** Z**:** the Q-Q plot is a curve ($\ln x + 5$ shape), not a straight line, so X and Z have **different distributions**. $Z = \ln X + 5$ is left-skewed (it can take negative values when $X < e^{-5}$, although rarely), while $X > 0$ is right-skewed. In the scatterplot all points lie exactly on the curve $z = \ln x + 5$: Z is a deterministic function of X, so they are **strongly dependent** (Spearman correlation $= 1$).

So the pair (X, Y) is identically distributed and independent, and the pair (X, Z) is dependent but not identically distributed.

2(a). X counts the Heads in 3 independent tosses of a fair coin, so it is **binomially distributed**: $X \sim \text{Bin}(3, 1/2)$, with

$$\mathsf{P}(X = k) = \binom{3}{k}\frac{1}{8}, \quad k = 0, 1, 2, 3 \quad \Rightarrow \quad \left(\tfrac{1}{8}, \tfrac{3}{8}, \tfrac{3}{8}, \tfrac{1}{8}\right).$$

It is not normal or Poisson (only 4 values, bounded), and not uniform (the probabilities differ).

2(b). $\mathsf{E}X = np = 1.5$ and $\text{Var}\, X = np(1-p) = 0.75$.

```
n_rounds = 100
xs = np.random.binomial(3, 0.5, size=n_rounds)
print(f"sample mean of X     = {xs.mean():.4f}   (theory 1.5)")
print(f"sample variance of X = {xs.var(ddof=1):.4f}   (theory 0.75)")
```

```
sample mean of X     = 1.3500   (theory 1.5)
sample variance of X = 0.6944   (theory 0.75)
```

The sample mean and sample variance are close to 1.5 and 0.75. With only 100 rounds the standard error of $\bar{X}$ is $\sqrt{0.75/100} \approx 0.087$, so deviations of about ± 0.17 are normal.

2(c). The friend's argument is **wrong**. They compute $(\mathsf{E}X)^2$, but the expected payoff is $\mathsf{E}(X^2)$, and as in 1(a), $\mathsf{E}g(X) \neq g(\mathsf{E}X)$ for nonlinear g. Since x^2 is convex, $\mathsf{E}X^2 \geq (\mathsf{E}X)^2$, and the gap is exactly $\operatorname{Var} X$:

$$\mathsf{E}Y = \mathsf{E}X^2 = \operatorname{Var} X + (\mathsf{E}X)^2 = 0.75 + 2.25 = 3.$$

Directly: $\mathsf{E}Y = 0 \cdot \frac{1}{8} + 1 \cdot \frac{3}{8} + 4 \cdot \frac{3}{8} + 9 \cdot \frac{1}{8} = \frac{24}{8} = 3$. The expected net gain is $3 - 3 = 0$, so **the game is fair**.

Next, $\mathsf{E}Y^2 = \mathsf{E}X^4 = 0 + 1 \cdot \frac{3}{8} + 16 \cdot \frac{3}{8} + 81 \cdot \frac{1}{8} = \frac{132}{8} = 16.5$, so

$$\operatorname{Var} Y = 16.5 - 3^2 = 7.5.$$

```
pmf = stats.binom.pmf(np.arange(4), 3, 0.5)
EY = np.sum(np.arange(4) ** 2 * pmf)
VarY = np.sum(np.arange(4) ** 4 * pmf) - EY ** 2
print(f"exact E(Y) = {EY:.4f},  Var(Y) = {VarY:.4f}")

ys = np.random.binomial(3, 0.5, size=n_rounds) ** 2
print(f"sample mean of Y     = {ys.mean():.4f}")
print(f"sample variance of Y = {ys.var(ddof=1):.4f}")
```

```
exact E(Y) = 3.0000,  Var(Y) = 7.5000
sample mean of Y     = 3.3500
sample variance of Y = 8.7753
```

The simulated $\bar{y}$ and s^2 are close to 3 and 7.5. Y has a large variance, so with 100 rounds $\bar{y}$ can easily be off by ± 0.5.

2(d). By the CLT, $\bar{Y} \approx \mathcal{N}(\mathsf{E}Y, \operatorname{Var} Y/n)$, so the 95% CI half-length is $\varepsilon = 1.96\, s/\sqrt{n}$.

```
s_y = ys.std(ddof=1)
eps_y = 1.96 * s_y / np.sqrt(n_rounds)
print(f"95% CI for E(Y): ({ys.mean() - eps_y:.3f}, {ys.mean() + eps_y:.3f}),"
      f"  half-length = {eps_y:.3f}")
print(f"(with the true sigma: half-length = {1.96 * np.sqrt(VarY) / np.sqrt(n_rounds):.3f})")

delta = 3 - 2.9
n_ci = int(np.ceil((1.96 * np.sqrt(VarY) / delta) ** 2))
n_power = int(np.ceil(((1.96 + stats.norm.ppf(0.8)) * np.sqrt(VarY) / delta) ** 2))
print(f"rounds for half-length < 0.1         : n >= {n_ci}")
print(f"rounds for a 5%-level test, 80% power: n >= {n_power}")
```

```
95% CI for E(Y): (2.769, 3.931),  half-length = 0.581
(with the true sigma: half-length = 0.537)
rounds for half-length < 0.1         : n >= 2882
rounds for a 5%-level test, 80% power: n >= 5887
```

Half-length of the CI. With our 100 simulated rounds the half-length is $\varepsilon = 1.96\, s/\sqrt{100} \approx 0.5$–$0.6$ (the printed value is 0.581; with the true σ it is $1.96\sqrt{7.5}/10 \approx 0.54$). The interval has width about 1, so it contains 3 but also many other values (e.g. 2.9): **the simulation alone cannot decide**

whether the game is fair. In our run the interval is $(2.769, 3.931)$, so it is consistent with a fee anywhere from about £2.8 to £3.9. A simulation never proves that $\mathsf{E}(G) = 0$; the exact computation in 2(c) does.

How many rounds to distinguish fee £3 from fee £2.9? The two games differ in the mean by $\delta = 0.1$, and "distinguish" can be made precise in two ways:

- *Criterion A (CI half-length).* Require $1.96\,\sigma/\sqrt{n} \le \delta$, i.e.

$$n \ge \left(\frac{1.96\sqrt{7.5}}{0.1}\right)^2 \approx 2882.$$

  This is only a **minimum**: it assumes that $\bar{Y}$ lands exactly on 3. Since $\bar{Y}$ is random, with $n = 2882$ the CI excludes 2.9 in only about **50%** of experiments.
- *Criterion B (test with 80% power).* We want the CI to exclude 2.9 in at least 80% of experiments, which gives

$$n \ge \left(\frac{(z_{0.975} + z_{0.8})\,\sigma}{\delta}\right)^2 = \left(\frac{(1.96 + 0.84)\sqrt{7.5}}{0.1}\right)^2 \approx 5900.$$

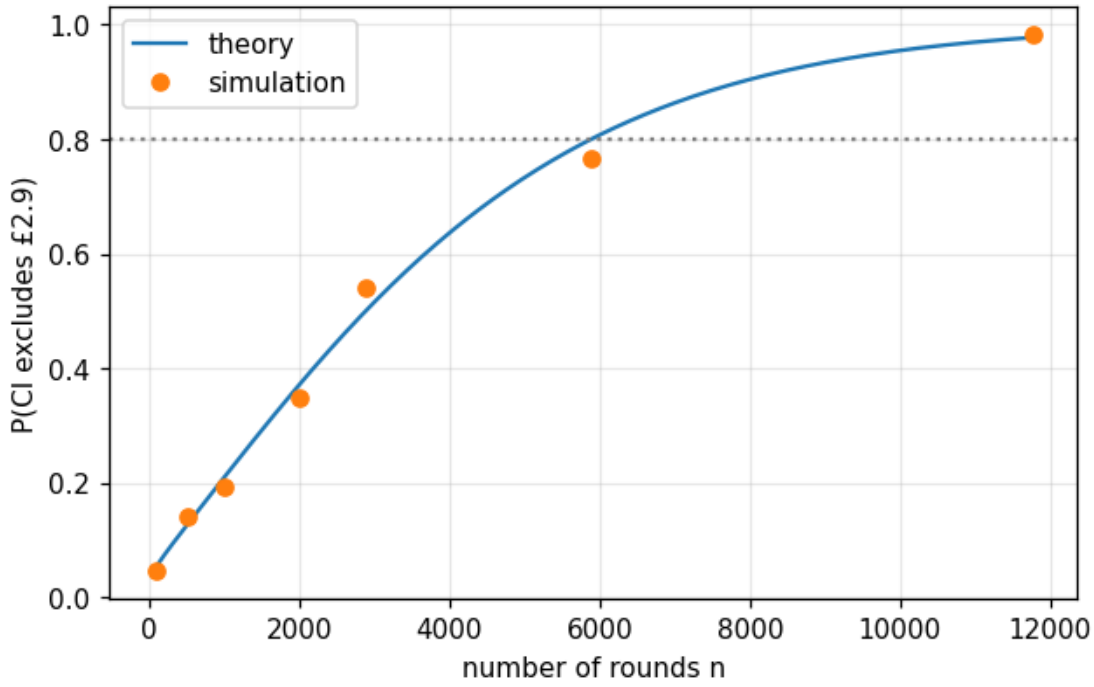
Answer: about 5900 rounds (criterion B); fewer than 2882 rounds cannot work at all. The simulation below confirms this: it repeats the whole experiment many times and counts how often the 95% CI for $\mathsf{E}(Y)$ excludes 2.9 (the true value is 3). The theoretical probability is $\Phi(\delta\sqrt{n}/\sigma - 1.96)$.

```
def share_excluding(value, n, R=300):
    Y = np.random.binomial(3, 0.5, size=(R, n)) ** 2
    half = 1.96 * Y.std(axis=1, ddof=1) / np.sqrt(n)
    return np.mean(np.abs(Y.mean(axis=1) - value) > half)

ns_check = np.array([100, 500, 1000, 2000, n_ci, n_power, 2 * n_power])
sim_p = np.array([share_excluding(2.9, m) for m in ns_check])
theo_p = stats.norm.cdf(0.1 * np.sqrt(ns_check) / np.sqrt(VarY) - 1.96)
print("   n    P(CI excludes 2.9): simulated   theory")
for m, s_, t_ in zip(ns_check, sim_p, theo_p):
    print(f"{m:6d}                     {s_:.3f}      {t_:.3f}")
print("sanity check (true value 3 is excluded, should be ~0.05):", share_excluding(3.0, n_power))

plt.figure(figsize=(6.5, 4))
grid = np.linspace(50, 2 * n_power, 400)
plt.plot(grid, stats.norm.cdf(0.1 * np.sqrt(grid) / np.sqrt(VarY) - 1.96), label="theory")
plt.plot(ns_check, sim_p, "o", label="simulation")
plt.axhline(0.8, color="gray", ls=":")
plt.xlabel("number of rounds n"); plt.ylabel("P(CI excludes £2.9)"); plt.legend(); plt.grid(alpha=
```

```
   n    P(CI excludes 2.9): simulated   theory
   100                     0.047      0.055
   500                     0.140      0.126
  1000                     0.193      0.210
  2000                     0.350      0.372
  2882                     0.540      0.500
  5887                     0.767      0.800
 11774                     0.983      0.977
sanity check (true value 3 is excluded, should be ~0.05): 0.05
```



At $n \approx 2882$ the CI excludes £2.9 in about half of the experiments, at $n \approx 5900$ in about 80%, and the simulation agrees with the theoretical curve. Hence roughly 5900 rounds are needed to distinguish the fair game (fee £3) from the game with fee £2.9 reliably.

General instructions/hints on implementation

1. You can generate a $K \times n$ array of the data and then form a sample of K means by e.g. using

```
s = np.random.exponential(scale=1/lam, size=(K, n)).mean(axis=1)
```

for the exponential distribution $\mathcal{E}(\lambda)$ with rate λ. Note that NumPy uses the scale (mean) $1/\lambda$, not the rate. Similarly, `np.random.poisson(mu, size=(K, n))` gives Poisson samples.

1. The e.c.d.f. F_S of the data vector `s` is

```
from scipy.stats import ecdf
Fs = ecdf(s).cdf.evaluate      # callable: Fs(t)
```

1. K must be at least 10000 to generate reliable e.c.d.f.'s of sample means.
2. `scipy.stats.norm(mu, sigma).cdf` gives the c.d.f. F_Z of $\mathcal{N}(\mu, \sigma^2)$. Note that the 2nd argument is the standard deviation σ.
3. To calculate the maximal difference between the two c.d.f.'s, take a grid of 200 points t_j within 3σ of μ and return $\max_{t_j}\{|F_S(t_j) - F_Z(t_j)|\}$:

```
t = np.linspace(mu - 3*sigma, mu + 3*sigma, 200)
np.max(np.abs(Fs(t) - Fz(t)))
```

1. To plot both c.d.f.'s on one graph, use `matplotlib.pyplot.step(t, Fs(t), where="post")` for the e.c.d.f. and `plt.plot(t, Fz(t))` for the normal c.d.f.
2. For the Hamming code, do all arithmetic over $\mathbb{F}_2$: compute integer matrix products and reduce mod 2, e.g. `c = (m @ G) % 2`, `z = (r @ H) % 2`. Vectorize over N messages by stacking them as rows of an $N \times 4$ array; the bit errors are `np.random.random((N, 7)) < p`.
3. For a Q-Q plot of two samples `x` and `y` of equal size, plot `np.sort(x)` against `np.sort(y)`; for a Q-Q plot against a theoretical distribution, use `scipy.stats.probplot`.
4. Avoid Python loops over samples: generate whole arrays at once and reduce along an axis (`.mean(axis=1)`, `.sum(axis=1)`). This is both faster and closer to the mathematical formulation.